

LLM 관련 학습 범위 (LLM API Programming or LLM system Engineering?)

개발환경은, OS는 windows, 프로그래밍은 파이썬으로 통일, IDE는 vs code, 파이썬 라이브러리는 numpy + PyTorch, PC 환경은 RTX5070 GPU 기반

- 1) LLM 자체에 대한 이해와 학습 (transformer 구조, attention mechanism, prefill/decode 단계, activation과 weights)
- 2) LLM interaction 및 chatting과 같은 서비스 개발 (이미 예제들이 많이 있고, 간단한 코드는 gemini.com 혹은 chatgpt.com, deepseek.com 같은 웹으로도 진행은 가능)
- 3) 특별히 Agent 가 무엇이고, 간단한 종류의 Agent SW를 개발해 나가는 과제 중심으로 학습
- 4) 내가 만들고자 하는 앱/서비스를 Gemini CLI 와 같은 coding agent에게 작업 지시하여, 점진적으로 개발해 나가는 과정 (바이브 코딩 기법)
- 5) LLM 관련 engineering 분야 - vLLM 과 같이 빠르고 효율적으로 LLM 인공지능 서비스를 운용시키는 다양한 서버 관련 기술 들.
- 6) NVIDIA GPU (CUDA library 포함) 를 본격 활용하는 개발 (linux 포함)

수업목표

- 1) 지루해 하지 않고, 원가 몰입하며 만들어가 가는 시간을 갖는다. (시간 때우기 식 수업이 되지 않도록)
- 2) 대단한 것을 만들지는 않아도 좋다. 집중하며 인공지능에 대한 무엇인가를 배울 수만 있다면.
- 3) 나중에 실습한 프로젝트를 포트폴리오에 추가할 수 있거나 교내 경진대회에 신청해 볼 수 있다면 금상첨화
- 4) 너무 대단한 것을 시도하는 것보다는 실속있게 하나 하나 배워 나가는 것이 좋지 않을까?

꽤 큰 규모의 개발을 할 경우의 check-points

- 무엇을 개발할 것인지 아이디어 그림을 가지고 있는지,
- 개발환경 차원에서 codebase (workspace)를 어디에 둘 것인지 (강의실 PC or 내 노트북 or 2개 Computer를 번갈아 가며 개발?)
- github를 잘 사용할 수 있는지
- GPU/CUDA 까지 상요한 본격 AI Programming 개발을 해보고 싶다면, WSL2 Ubuntu linux 필요.

아래는 LLM 및 Python 개발에 익숙하지 않은 학생들을 대상으로, 난이도에 따라 단계적으로 팀 프로젝트를 설계하기 위한 사전 점검(Check-list) 체계를 재구성한 것입니다. 현재 수업 구조(Windows + VS Code + PyTorch + RTX GPU, 일부 팀은 Ubuntu + L40S 서버 활용)를 반영하여 기초 → 응용 → Agent → 서비스화 → 엔지니어링 → GPU 심화의 6단계 학습 경로에 맞추어 정리합니다.

① 전체 교육 로드맵 (난이도 기반 6단계 구조)

단계	학습 수준	핵심 목표	팀 프로젝트 난이도
Level 0	개발환경 적응	Python + GPU 환경 세팅	매우 낮음
Level 1	LLM 이해	Transformer 구조 이해	낮음

Level 2	LLM 활용	Chat / API 기반 서비스	낮음~중간
Level 3	Agent 개발	Tool + Loop 기반 Agent	중간
Level 4	서비스 자동화	Coding Agent 활용 개발	중간~상
Level 5	LLM Engineering	vLLM / 서버 운영	상
Level 6	CUDA 심화	GPU 병렬 프로그래밍	최상

② 단계별 사전 점검 사항 (Pre-flight Checklist)

◆ Level 0 – 개발환경 점검 (모든 팀 공통 필수)

✓ 기술 준비도 체크

- Windows + VS Code 정상 동작
 - Python 가상환경 생성 가능 (venv or conda)
 - numpy + PyTorch 설치 성공
 - `torch.cuda.is_available()` 확인
 - `nvidia-smi` 실행 확인
 - GitHub clone / push 가능
 - WSL2 Ubuntu 설치 여부 (GPU 심화팀 필수)
-

◆ Level 1 – LLM 기본 구조 이해

✓ 이론 이해 점검

- Transformer에서 Self-Attention이 하는 역할 설명 가능?
- Query / Key / Value 의미 설명 가능?
- 파라미터 수가 많다는 것이 무엇을 의미하는가?
- PyTorch로 간단한 matmul 구현 가능?

✓ 실습 체크

- PyTorch로:
 - 단순 Linear layer 구현
 - autograd 사용
 - 작은 attention toy example 구현

목표: “LLM은 거대한 행렬연산 시스템”임을 이해하게 하는 단계

◆ Level 2 – LLM Interaction 기반 앱 개발

✓ 기본 역량 점검

- LLM API 호출 가능 (Ollama or OpenAI API 형식)
- prompt 설계 가능
- JSON 응답 파싱 가능
- Python 함수 구조 이해

✓ 팀 프로젝트 난이도 (초급)

- 간단한 챗봇
- 문서 요약기
- 번역기
- 키워드 분석기

이 단계는 웹 없이도 **CLI** 기반으로 충분

◆ Level 3 – Agent 개념 도입

✓ 반드시 이해해야 할 것

- Agent = LLM + Memory + Tool + Loop
- Thought → Action → Observation 구조
- Tool calling 구조
- 상태(state) 유지

✓ 체크 질문

- Agent와 단순 Chatbot의 차이?
- 함수 호출 기반 reasoning이 왜 필요한가?
- Multi-step reasoning이 필요한 상황 예시는?

✓ 팀 프로젝트 (중간 난이도)

- 날씨 API Agent
- 일정 관리 Agent
- 파일 분석 Agent
- RAG 기반 질의응답 Agent

이 단계에서 처음으로 “시스템 설계” 개념 도입

◆ Level 4 – Coding Agent 활용 개발 (Vibe Coding)

여기서부터 난이도 급상승

✓ 사전 점검

- Git 브랜치 전략 이해?
- 모듈 분리 가능?
- requirements.txt 관리 가능?
- 코드 리뷰 가능?

✓ 학습 목표

- 내가 만들 앱 구조 설계
- Gemini CLI 같은 coding agent에게 작업 지시
- 점진적 리팩토링
- 기능 단위 개발

이 단계는 기획 능력 + 구조 설계 능력이 핵심

◆ Level 5 – LLM Engineering (서버 트랙 전용)

여기서부터는 일반 앱 개발이 아니라 “AI 시스템 운영”

✓ 사전 필수

- Linux 기본 명령어 숙련
- 포트 / IP / 서버 구조 이해
- Docker 개념 이해
- 프로세스 vs 스레드 이해

✓ 기술 요소

- vLLM
- Kubernetes
- KServe
- Multi-user serving
- 모델 메모리 관리
- Throughput vs Latency 이해

이 단계는 2개 팀 정도만 수행 권장

◆ Level 6 – CUDA / GPU 심화

이 단계는 연구지향 학생 대상

✓ 사전 점검

- 행렬 연산 복잡도 이해
- 메모리 계층 (Global / Shared / Register) 이해

- CUDA thread block 구조 이해

✓ 실습

- PyTorch custom CUDA extension
- 간단한 kernel 작성
- GEMM 성능 비교
- vLLM 내부 구조 분석

이 단계는 “LLM을 쓰는 사람”이 아니라
“LLM을 빠르게 만드는 사람” 단계

③ 대규모 프로젝트 사전 체크 포인트 (팀 단위)

1. 문제 정의 명확성

- 무엇을 만들 것인가?
- 사용자는 누구인가?
- 단순 Chatbot인가, Agent인가?
- 외부 API 필요한가?
- 멀티 유저인가?

👉 “아이디어 그림(Architecture Diagram)” 제출 필수

2. 개발 환경 전략

- 코드베이스 위치:
 - 강의실 PC?
 - 개인 노트북?
 - GitHub 중앙 관리?
 - GPU 자원 공유 전략?
 - 서버 접속 방식?
-

3. Git 운영 체크

- PR 사용 가능한가?
 - merge conflict 해결 가능?
 - branch 전략 수립했는가?
 - 팀원 간 역할 분담 명확한가?
-

4. GPU 활용 의지 점검

다음 질문에 “Yes”면 GPU 심화팀 가능:

- Linux 환경이 불편하지 않은가?
- 에러 로그 분석을 두려워하지 않는가?
- 환경 설정 1~2시간 삼질 가능?
- 메모리 에러 대응 경험하고 싶은가?

4 팀 수준별 권장 프로젝트 매칭

팀 수준	추천 프로젝트
매우 초급	LLM 기반 문서 요약 서비스
초중급	RAG 기반 Q&A
중급	Tool 기반 Agent
중상급	Multi-agent 시스템
상급	vLLM multi-user 서버
연구지향	CUDA 커널 성능 최적화

LLM Programming/Engineering 수업은 보통 다음 두 부류로 나뉩니다:

1. “API 호출 수업” → 난이도 낮음, 금방 끝남
2. “시스템 설계 수업” → 난이도 높음, 제대로 남음

7 권장 수업 구조 (4주 집중형 예시)

Week 1: 환경 세팅 + LLM 기초 + PyTorch 실습

Week 2: Agent 구조 구현

Week 3: 웹 서비스화

Week 4: 고급팀: vLLM / 서버 운영, 일반팀: 기능 고도화